Hundir la flota

The Bridge | Data Science Bootcamp

Jorge Sánchez Hernández



Estructura del proyecto

utils.py numpy, random

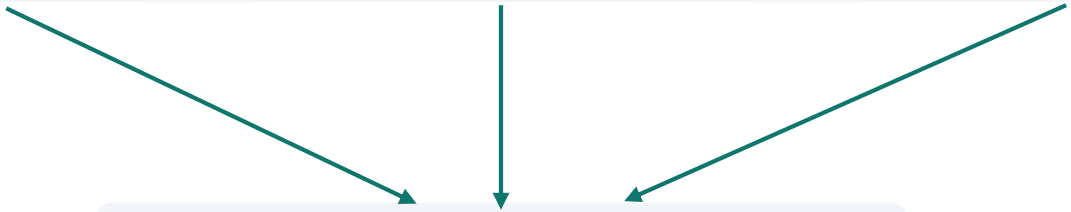
Tableros, barcos, disparos y vidas

bot.py random

La CPU y sus 3 niveles

visual.py time, sys

Slowprint: texto lento en la visualización del terminal



main.py numpy, random

Bucle de la partida y turnos

main.py importa funciones de los otros tres módulos para el correcto funcionamiento de la partida

Leyenda: El tablero y la flota.

Agua sin disparar

Casilla vecina a un barco (Máscara)

Agua disparada

Barco tocado

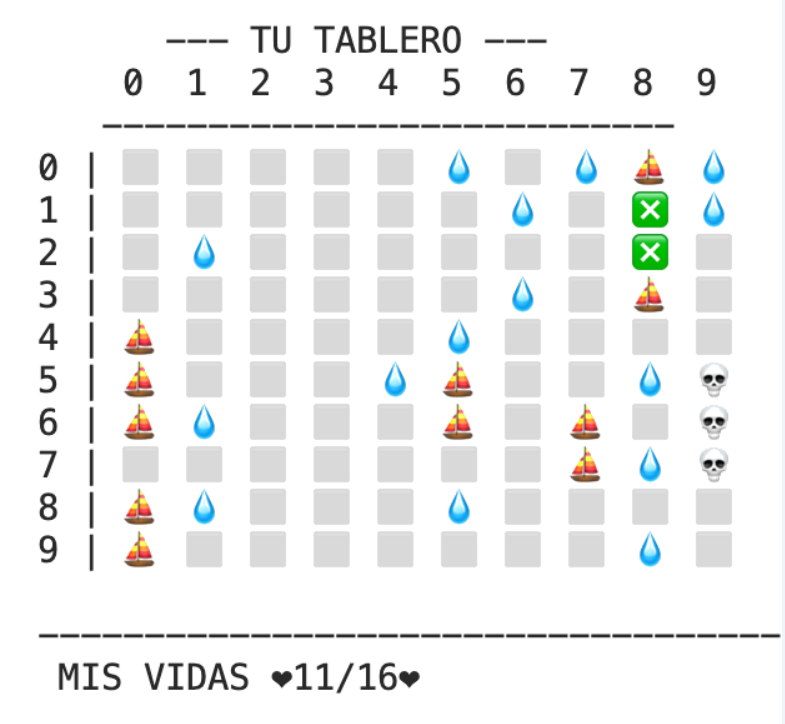
Barco hundido

Barco no tocado

utils.py (create_fleet)

```
fleet_lengths = [4,3,3,2,2,2]
```

Terminal



utils.py · Crear el tablero y los barcos

create_board

Matriz 10×10 llena de ' '

create_board

```
def create_board(size=10):
  return np.full(
    (size, size), ' ')
```

create_ship

Elige orientación y posición al azar y evita que no choque con otros Barcos ni que se salga del tablero

create_ship

```
def create_ship(length, board):
    size = 10

    while True:
        orientation = random.choice
            (['H', 'V'])

        if orientation == 'H':
            row = np.random.randint
              (0, size)
            column = np.random.randint
              (0, size - length + 1)
        else:
            row = np.random.randint
              (0, size - length + 1)
            column = np.random.randint
              (0, size)
```

```
ship = []

    for i in range(length):
        if orientation == 'H':
            ship.append((row, column + i))
        else:
            ship.append((row + i, column))

    collision = False
    for row, column in ship:
        if board[row, column] != ' ':
            collision = True
            break

    if not collision:
        return ship
```

utils.py · Colocación de barcos

place_ship Rodea el barco de ⬛ y pone ⛵ por cada eslora

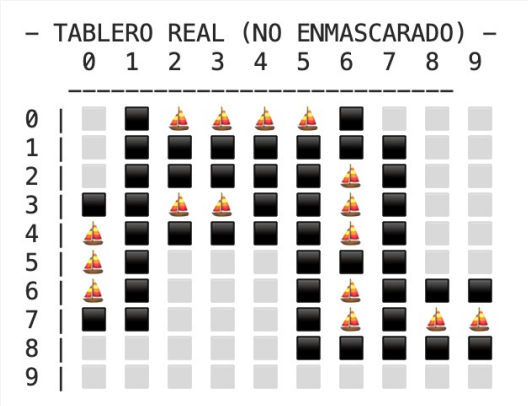
place_ship

```
def place_ship (ship, board):
  size = 10
  boxes_around = [(-1, -1), (-1, 1), (1, -1), (1, 1)
                  , (0,1), (1,0), (-1, 0), (0, -1)]

  for row, col in ship:
     for row_around, col_around in boxes_around:
          row_mask, col_mask = row_around + row, col_around + col
          if 0 <= row_mask < size and 0 <= col_mask < size:
              if board[row_mask, col_mask] == '  ':
                  board[row_mask, col_mask] = '⬛'

  for row, column in ship:
      board[row, column] = '⛵'
  return board
```

Terminal



create_fleet

Recorre [4,3,3,2,2,2]: crea, coloca y guarda cada barco en fleet_list.

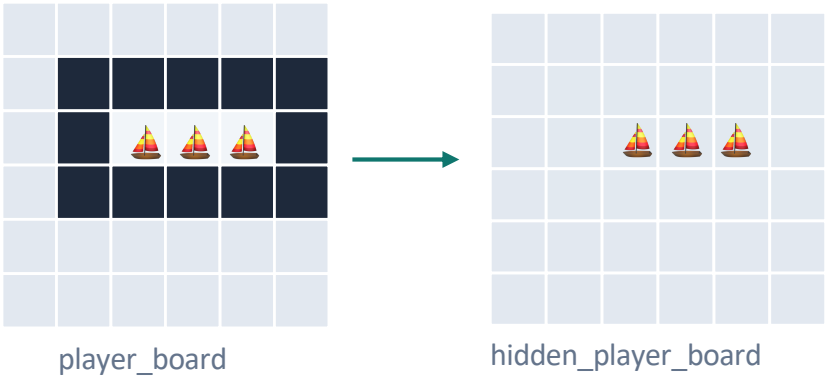
create_fleet

```
def create_fleet (board):

      fleet_lenghts = [4,3,3,2,2,2]
      fleet_list = []
      for lenght in fleet_lenghts:
        new_ship = create_ship(lenght, board)
        board = place_ship(new_ship, board)
        fleet_list.append(new_ship)

      return board, fleet_list
```

utils.py · Crear máscara para no ver tablero enemigo



hidden

```
def hidden_board_cpu(board):
    hidden = board.copy()
    hidden[hidden =='⛵'] = '  '
    hidden[hidden == '🟧'] = '  '
    return hidden

def hidden_board_player(board):
    hidden= board.copy()
    hidden[hidden == '🟧'] = '  '
    return hidden

def lost_board_cpu(board):
    hidden= board.copy()
    hidden[hidden == '🟧'] = '  '
    return hidden
```

utils.py · shoot

1

Coordenadas

Input (0 al 9)
- Fila
- Columna

shoot

```
def shoot(board, fleet_list, box=None):

if box is None:
        while True:
                try:
                    row = int(input("Escoge una fila del 0 al 9:"))
                    if 0 <= row <= 9:
                        break
                    else:
                        slowprint("⚠ ERROR ⚠  Introducir un número del 0 al 9")
                except ValueError:
                    slowprint("⚠ ERROR ⚠  Introducir un número del 0 al 9")

        while True:
                try:
                    column = int(input("Escoge una columna del 0 al 9:"))
                    if 0 <= column <= 9:
                          break
                    else:
                          print("⚠ ERROR ⚠  Introducir un número del 0 al 9")
                 except:
                    if ValueError:
                           print("⚠ ERROR ⚠  Introducir un número del 0 al 9")

       box = (row, column )
 row, column = box
```

utils.py · shoot

2

¿Hay ⛵?

Pasa a ❎ (tocado)

3

¿Hundido?

Si no quedan ⛵,
todo el barco pasa a 💀

shoot

```
if board[row, column] == '⛵':
        board[row, column] = '❎'

        shipwreck = True

        for ship in fleet_list:
            if (row, column) in ship:
                for row_ship, column_ship in ship:
                    if board[row_ship, column_ship] == '⛵':
                        shipwreck = False
                        break
                if shipwreck:
                    for row_ship, column_ship in ship:
                        board[row_ship,column_ship] = '💀'
                break

        if shipwreck == False:
            slowprint(f"\nDisparo en ({row}, {column}): TOCADO 💣.✷")
        else:
            slowprint(f"\nDisparo en ({row}, {column}): 💥 TOCADO Y HUNDIDO!!!")
```

utils.py · shoot

4

Agua

⬜/⬛ pasan a 💧

5

Repetido

“Ya has disparado”

```
elif board[row, column] in ('⬜', '⬛'):
        board[row, column] = '💧'
        slowprint(f"\nDisparo en ({row}, {column}): AGUA ★。°🌊。")

    else:
        slowprint(f"\nYa has disparado en ({row}, {column})")

    return board
```

utils.py · shoot /lives

4

Agua

⬜/⬛ pasan a 💧

5

Repetido

“Ya has disparado”

shoot

```
elif board[row, column] in ('⬜', '⬛'):
        board[row, column] = '💧'
        slowprint(f"\nDisparo en ({row}, {column}): AGUA ★。˚🌊。")

    else:
        slowprint(f"\nYa has disparado en ({row}, {column})")

    return board
```



lives

Cuenta los ⛵ que quedan con np.sum. Llega a 0 = flota hundida.

lives

```
def  lives(board):
    remaining_lives = np.sum(board == '⛵')
    return remaining_lives
```

print_boards - Interfaz partida

print_boards

```
def print_boards(player_board, cpu_board):
    import sys
    import time

    sys.stdout.write("\n📡 Escaneando tablero")
    sys.stdout.flush()
    for _ in range(3):
        time.sleep(0.3)
        sys.stdout.write(".")
        sys.stdout.flush()

    time.sleep(0.2)
    cpu_hidden_board = hidden_board_cpu(cpu_board)
    player_hidden_board = hidden_board_player(player_board)

    print("\n      --- TU TABLERO ---                  --- TABLERO ENEMIGO ---")
    print("    0  1  2  3  4  5  6  7  8  9           0  1  2  3  4  5  6  7  8  9")
    print("   ---------------------------             ---------------------------")

    for r in range(10):
        player_row = " ".join(player_hidden_board[r])
        cpu_row = " ".join(cpu_hidden_board[r])
        print(f"{r} | {player_row}       {r} | {cpu_row}")

        time.sleep(0.12)

    lives_player = lives(player_board)
    lives_cpu = lives(cpu_board)

    print(f" MIS VIDAS ❤️{lives_player}/16❤️                                VIDAS ENEMIGO ❤️{lives_cpu}/16❤️")
```

Terminal

```
======================TURNO 1================================

📡 Escaneando tablero...


      --- TU TABLERO ---                --- TABLERO ENEMIGO ---
    0  1  2  3  4  5  6  7  8  9          0  1  2  3  4  5  6  7  8  9
   ---------------------------          ---------------------------
0 |  |  |  |  | ⛵| ⛵| ⛵|  |  |  |  0 |  |  |  |  |  |  |  |  |  |  |
1 |  |  |  |  |  |  |  |  |  |  |  1 |  |  |  |  |  |  |  |  |  |  |
2 |  |  |  |  |  |  | ⛵|  |  |  |  2 |  |  |  |  |  |  |  |  |  |  |
3 |  |  |  |  |  |  | ⛵|  |  |  |  3 |  |  |  |  |  |  |  |  |  |  |
4 |  |  |  |  |  |  |  |  | ⛵|  |  4 |  |  |  |  |  |  |  |  |  |  |
5 |  |  | ⛵|  |  |  |  |  | ⛵|  |  5 |  |  |  |  |  |  |  |  |  |  |
6 |  |  | ⛵|  |  |  | ⛵|  | ⛵|  |  6 |  |  |  |  |  |  |  |  |  |  |
7 |  |  |  |  |  |  | ⛵|  |  |  |  7 |  |  |  |  |  |  |  |  |  |  |
8 |  | ⛵| ⛵| ⛵| ⛵|  |  |  |  |  |  8 |  |  |  |  |  |  |  |  |  |  |
9 |  |  |  |  |  |  |  |  |  |  |  9 |  |  |  |  |  |  |  |  |  |  |

------------------------------------------------------------------------------
 MIS VIDAS ❤16/16❤                                VIDAS ENEMIGO ❤16/16❤


 TU TURNO de disparar 💣

---------🔫 Es TU TURNO de disparar 🔫 -------
Escoge una fila del 0 al 9:7
Escoge una columna del 0 al 9:H
⚠️ERROR ⚠️ Introducir un número del 0 al 9
Escoge una columna del 0 al 9:9.4
⚠️ERROR ⚠️ Introducir un número del 0 al 9
Escoge una columna del 0 al 9:4

Disparo en (7, 4): AGUA ⋆｡˚🌊。
```

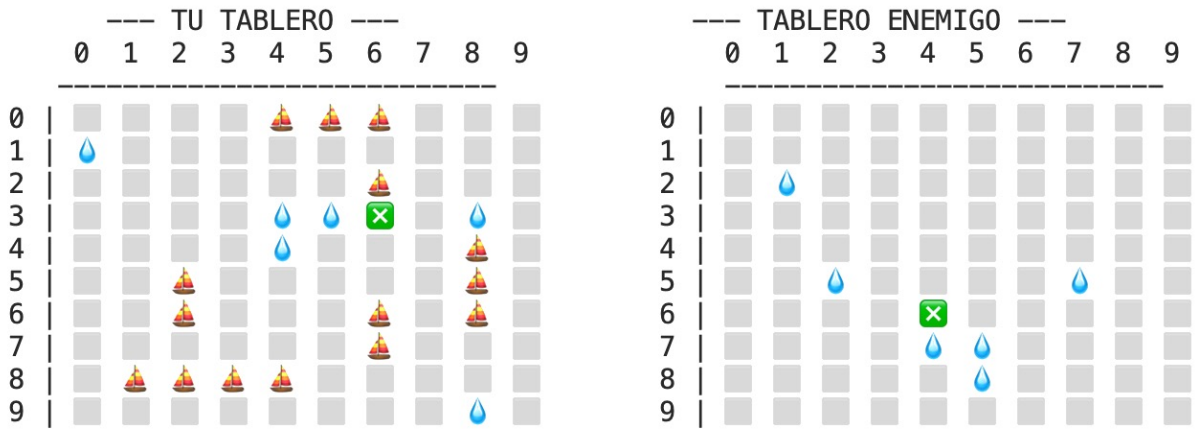
Terminal

```
 -------TURNO de la CPU-------- 🤖

Disparo en (3, 4): AGUA ⋆｡˚🌊。

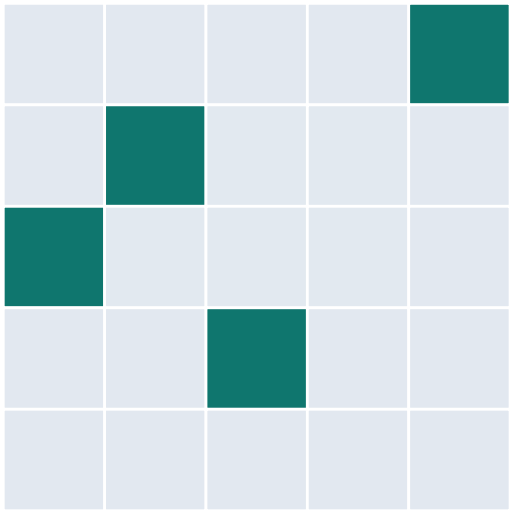
======================TURNO 8================================

📡 Escaneando tablero...
```

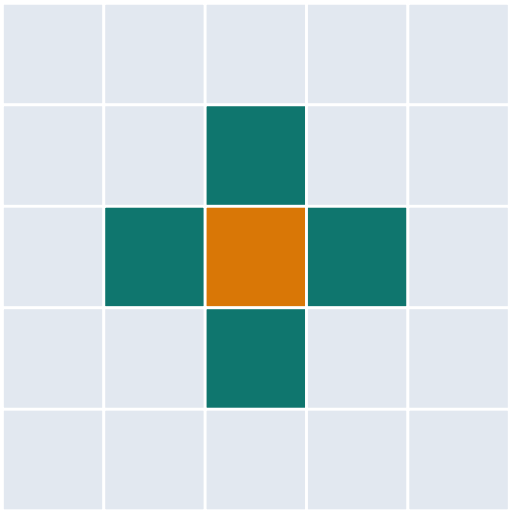



```
------------------------------------------------------------------------
 MIS VIDAS ❤15/16❤                          VIDAS ENEMIGO ❤15/16❤
```

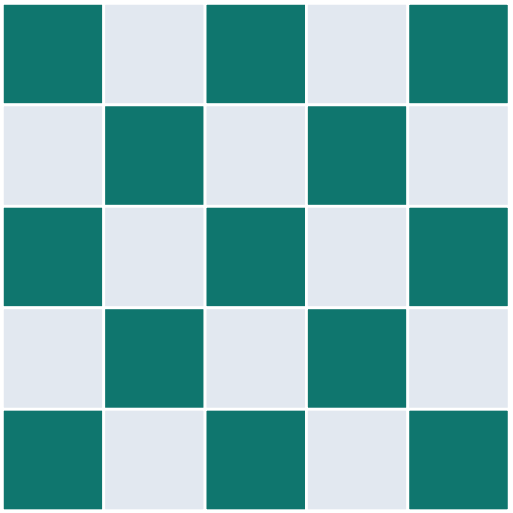
bot.py · Niveles de la cpu



Easy: Aleatorio



Medium: tras un ❎ dispara a los 4 vecinos. Sin ❎ → Easy



Hard: sin ❎ pendientes, dispara en damero. Tras un ❎ → Medium

bot.py · easy cpu

__init__

Guarda size y una paridad aleatoria (0/1).

easy_cpu

Dispara al azar
Devuelve None si no quedan casillas

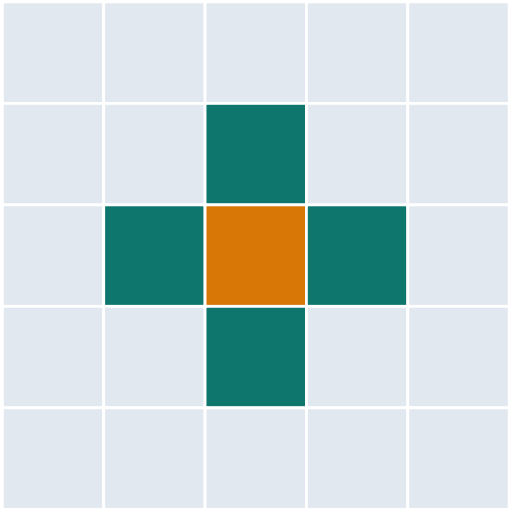
```
class cpu:
def __init__(self, size = 10):
    self.size = 10
    self.parity = random.choice([0,1])
```

class cpu

```
def easy_cpu(self, player_board):
random_shots = []
        for rows in range (self.size):
            for columns in range (self.size):
                if player_board [rows, columns] == '  ' or player_board [rows, columns] == '⛵':
                    random_shots.append((rows, columns))

        if len(random_shots) > 0:
            return random.choice(random_shots)
        else:
            return None
```

bot.py · medium cpu



Medium: tras un ❎ dispara a los 4 vecinos. Si no hay ❎ → Easy.cpu

medium_cpu

```
def medium_cpu(self, player_board):
    hit_boxes = []
    for hit_rows in range(self.size):
        for hit_columns in range(self.size):
            if player_board[hit_rows, hit_columns] == '❎':
                hit_boxes.append((hit_rows, hit_columns))

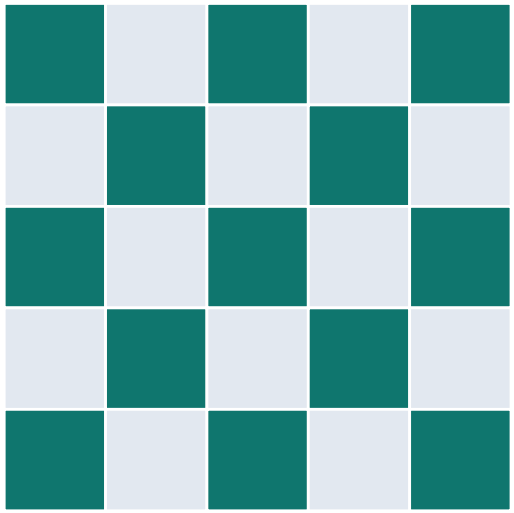
    for (hit_rows, hit_columns) in hit_boxes:
        boxes_around = [(0,1), (1,0), (-1,0), (0,-1)]
        for around_rows, around_columns in boxes_around:
            new_rows = around_rows + hit_rows
            new_columns = around_columns + hit_columns

            if (new_rows >= 0 and new_rows < self.size and
                new_columns >= 0 and new_columns < self.size):

                if (player_board[new_rows, new_columns] == '⬜' or
                    player_board[new_rows, new_columns] == '⛵'):
                    return (new_rows, new_columns)

    return self.easy_cpu(player_board)
```

bot.py · hard cpu



Hard: sin ❎ pendientes, dispara con un patron de tablero de ajedrez

Tras un ❎ → Medium

hard_cpu

```
def hard_cpu(self, player_board):
    hit_boxes = []
    for hit_rows in range(self.size):
        for hit_columns in range(self.size):
            if player_board[hit_rows, hit_columns] == '❎':
                hit_boxes.append((hit_rows, hit_columns))

    if len(hit_boxes) > 0:
        return self.medium_cpu(player_board)

    pair_boxes = []
    for pair_rows in range(self.size):
        for pair_columns in range(self.size):
            pair_box = player_board[pair_rows, pair_columns]
            if pair_box in ('  ', '⛵'):
                if (pair_rows + pair_columns) % 2 == self.parity:
                    pair_boxes.append((pair_rows, pair_columns))

    if len(pair_boxes) > 0:
        return random.choice(pair_boxes)

    return self.easy_cpu(player_board)
```

main.py · play() – Configuración de la partida

1

Inicio

Bienvenida y elección de nivel validada

2

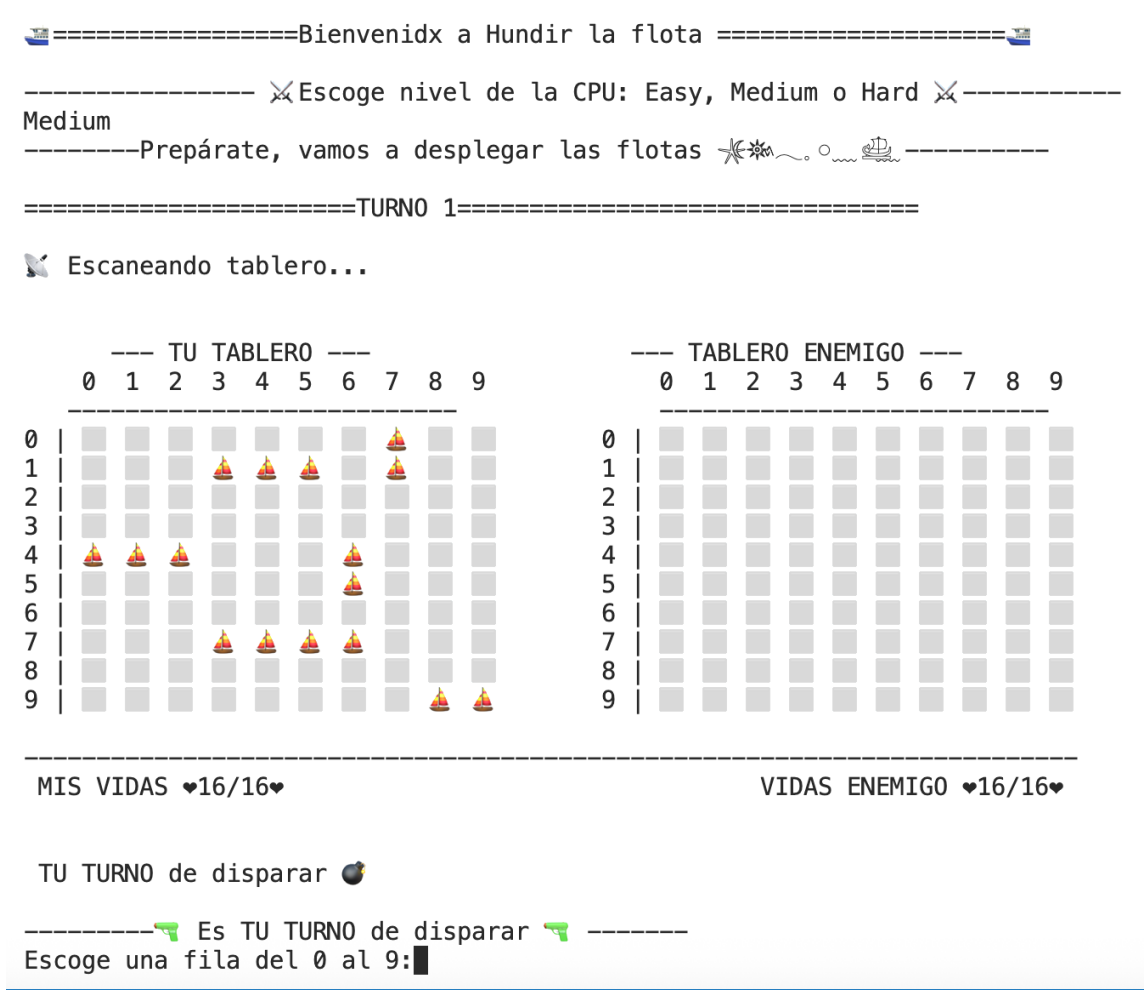
Setup

Dos tableros y dos flotas con create_fleet

main.py

```
def play():

    dificulty = input(Escoge nivel de la CPU: Easy, Medium o Hard ⚔\n").strip().lower()
    while dificulty not in ["easy","medium", "hard"]:
        slowprint("Introduce una dificultad válida")
        dificulty = input(Escoge nivel de la CPU: Easy, Medium o Hard ⚔).strip().lower()
    slowprint("Prepárate, vamos a desplegar las flotas")
    player_board = create_board()
    cpu_board = create_board()
    player_board, player_fleet = create_fleet(player_board)
    cpu_board, cpu_fleet = create_fleet(cpu_board)
    game_over = False
    turn = 1
```


Bienvenidx a Hundir la flota
Escoge nivel de la CPU: Easy, Medium o Hard
Medium
Prepárate, vamos a desplegar las flotas
TURNO 1
Escaneando tablero...
TU TABLERO
TABLERO ENEMIGO
0 1 2 3 4 5 6 7 8 9
MIS VIDAS ❤16/16❤
VIDAS ENEMIGO ❤16/16❤
TU TURNO de disparar
Es TU TURNO de disparar
Escoge una fila del 0 al 9:

main.py · play() – Bucle del juego

3

Tu turno: Dispara la casilla elegida y comprueba las vidas de la CPU

play()

```
…
while game_over == False:
        print(f"\n======================TURNO {turn}================================")
        print_boards(player_board, cpu_board)
        slowprint("\n TU TURNO de disparar 💣")
        cpu_board = shoot(cpu_board, cpu_fleet)
        if lives(cpu_board) == 0:
            print_boards(player_board, cpu_board)
            slowprint ("\n🏆🏆🏆 ENHORABUENA!! HAS HUNDIDO LA FLOTA ENEMIGA 🏆🏆🏆")
            break
```

main.py · play() – Bucle del juego

4

Turno de la CPU: Dispara la casilla elegida y comprueba las vidas de la CPU

play() …(while)

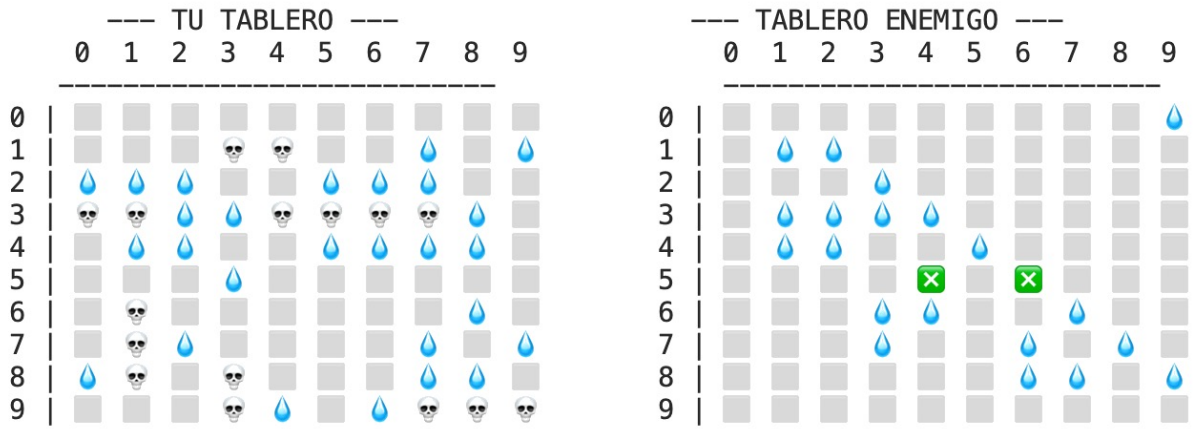
```
cpu_level = {"easy": bot.easy_cpu, "medium": bot.medium_cpu, "hard": bot.hard_cpu}
player_mask_board = hidden_board_player(player_board)
player_mask_board[player_mask_board == '■'] = ' '
shot_box = cpu_level[dificulty](player_mask_board)
if shot_box is not None:
    player_board = shoot(player_board, player_fleet, box = shot_box)
else:
    print("La CPU ya no tiene más casillas a las que disparar")
if lives(player_board) == 0:
    print_boards(player_board, cpu_board)
    slowprint(f"\n☠ ¡HAS SIDO DERROTADO)
    slowprint(f"Revelando la posición de la flota enemiga...")
    cpu_lost_board = lost_board_cpu(cpu_board)
    game_over = True
    break
 turn += 1
```

Terminal

-------TURNO de la CPU-------- 🤖

Disparo en (8, 3): 💥 TOCADO Y HUNDIDO!!!

📡 Escaneando tablero...

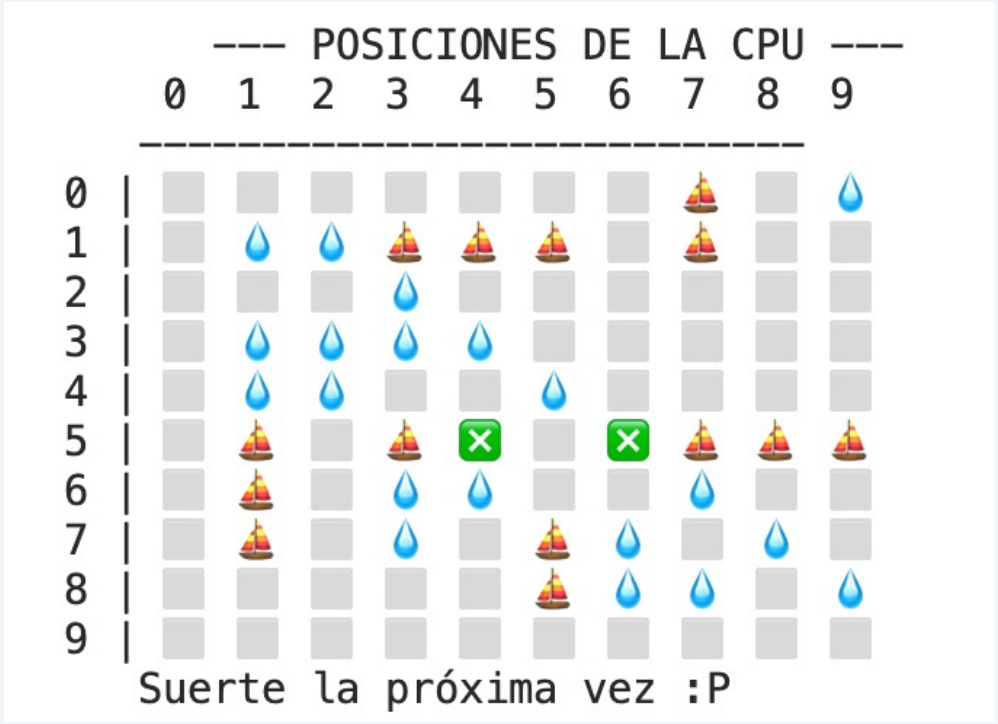


MIS VIDAS ❤0/16❤ VIDAS ENEMIGO ❤14/16❤

☠️ ¡HAS SIDO DERROTADO en el TURNO 43! Todos tus barcos se han hundido ☠️
Revelando la posición de la flota enemiga...

☠️ ¡HAS SIDO DERROTADO en el TURNO 43! Todos tus barcos se han hundido ☠️
Revelando la posición de la flota enemiga...

Terminal



Terminal



jorgeshz@MacBook-Air-de-Jorge Hundir_La_Flota %

¡Muchas gracias!

The Bridge | Data Science Bootcamp

Jorge Sánchez Hernández

